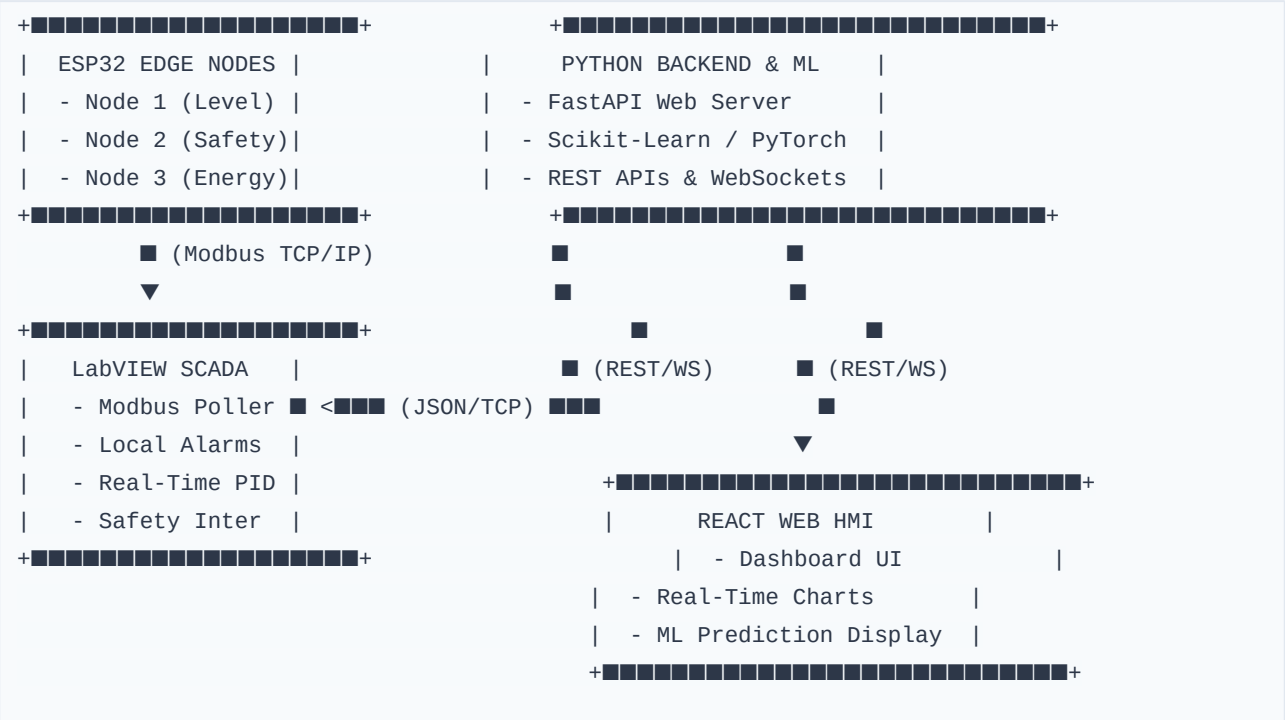


Project Guide: Advanced Industry 4.0 Distributed SCADA

Architecture: ESP32 + LabVIEW + Python Machine Learning + React HMI

This guide outlines the system architecture, component stack, implementation phases, work division, and communication structures for your distributed SCADA project.

1. Full System Architecture



How the Data Flows:

- Edge Acquisition:** ESP32 nodes gather physical sensor data and serve it over Wi-Fi via **Modbus TCP/IP**.
- SCADA Core (LabVIEW):** LabVIEW polls the ESP32s every 200ms, manages safety-critical PID control loops, and handles emergency shutdowns.
- Data Bridging:** LabVIEW packages the live readings into a JSON string and sends a POST request to the **Python Backend** (running locally or on a server).
- Machine Learning & Storage:** Python processes the incoming data, runs it through an **ML Model** (predicting faults, forecasting sensor values, or classifying status), and saves the results.
- Modern Visualization:** The **React HMI** dashboard communicates with the Python Backend via WebSockets or REST APIs to render high-fidelity, real-time graphs and predictions.

2. Technical Stack Details

A. Python Backend & ML (The Intelligence Layer)

- **Web Framework: FastAPI.** It is fast, automatically generates documentation, and natively supports WebSockets.
- **Machine Learning Tasks:**
- **Classification (Anomaly Detection):** A Random Forest or SVM model trained to classify system health (e.g., 0 = Normal, 1 = Warning: Sensor Drift, 2 = Critical: Gas Leak / Pump Cavitation).
- **Regression (Predictive Maintenance):** A linear regression or LSTM model that analyzes the temperature/power consumption over the last 10 seconds to predict the time-to-failure or future temperature spikes.
- **Libraries:** scikit-learn (for ML), numpy / pandas (for data processing), uvicorn (to run FastAPI).

B. React Web HMI (The Visual Layer)

- **Build Tool: Vite** (extremely fast development and build times).
- **Styling: Tailwind CSS** (for rich, responsive dark-mode aesthetics).
- **UI Components: Recharts** or **Chart.js** (for smooth, animated, real-time telemetry line graphs) and **Lucide React** (for sleek industrial icons).
- **Networking:** Standard JavaScript WebSocket or Axios to fetch data from the Python FastAPI backend.

3. Phase-by-Phase Integration Plan

Phase 1: Local Firmware & LabVIEW Engine (Weeks 1-3)

- **Goal:** Set up the physical hardware (ESP32, sensors, relays) and get LabVIEW polling them via Modbus TCP/IP.
- **LabVIEW Output:** Create a loop in LabVIEW that formats the current data as a JSON string: `json { "tank_level": 45.2, "temperature": 28.6, "gas_ppm": 120, "current_draw": 1.25, "timestamp": "2026-07-20T16:52:00" }` And sends it to `http://localhost:8000/api/telemetry` using LabVIEW's HTTP Client VIs.

Phase 2: Python Backend & Machine Learning (Weeks 4-6)

- **Goal:** Create the FastAPI server and develop the ML models.
- **Step 1: Train the ML Model.**
- Gather training data by running the system in different scenarios (normal filling, heating, introducing artificial "leaks" or covering the gas sensor).
- Train a classification model in Python (using Jupyter Notebooks/Scikit-learn) on this dataset.
- Save the trained model to disk as a `.pkl` file.
- **Step 2: Build the API.**
- Write a FastAPI app that loads the `.pkl` model.
- Create a POST endpoint `/api/telemetry` where LabVIEW sends data. When data arrives, run `model.predict(data)` and output the result.
- Create a WebSocket endpoint `/ws/live` that broadcasts the sensor data and ML prediction to any connected React clients.

Phase 3: React Web Frontend (Weeks 7-8)

- **Goal:** Create a modern, responsive web app that serves as your SCADA HMI.
- **Design Guidelines:**
 - Create a dark-themed layout with glassmorphic cards (semi-transparent backdrops with subtle borders).
 - Use **Recharts** to draw a live sliding graph of the water level and temperature.
 - Add a prominent "System Health Status" badge. It should dynamically change color based on the Python ML prediction:
 - Green: **HEALTHY (Class 0)**
 - Yellow: **MAINTENANCE REQUIRED (Class 1)**
 - Red: **SYSTEM FAULT / CRITICAL (Class 2)**
- Add control switches that send API commands back to Python (which forwards them to LabVIEW) to override relays.

Phase 4: Integration, Validation, and Calibration (Weeks 9-10)

- Connect all components together.
- Simulate a sensor failure (e.g., disconnect a sensor pin) and verify that the Python ML engine immediately classifies it as a fault and that the React interface flashes red.
- Record latency: Measure how long it takes for a physical sensor reading to travel through ESP32 -> LabVIEW -> Python (ML) -> React. (Usually under 100-200ms locally).

5. Realigned Team Roles (3-Member Division)

Because the project now spans Embedded C++, LabVIEW, Python, and JavaScript, work division must be highly structured:

Member	Primary Focus	Technical Stack	Key Deliverables
Member 1 (Embedded & LabVIEW)	- Physical wiring & ESP32 RTUs. - LabVIEW Modbus TCP polling. - LabVIEW HTTP JSON client.	- Arduino C++ - NI Modbus Library - LabVIEW HTTP VIs	- Functioning physical plant. - JSON telemetry sent out from LabVIEW.
Member 2 (Python & Machine Learning)	- Training classification/regression models. - Creating the FastAPI server. - Database logging & API endpoints.	- Python 3.x - Scikit-learn / Pandas - FastAPI & Uvicorn	- Trained ML model (.pkl file). - Working FastAPI server with REST/WebSockets.

Member 3 (Web HMI Developer)	- Creating the React web dashboard. - Designing responsive Tailwind UI. - Connecting WebSockets for real-time charting.	- React.js (Vite) - Tailwind CSS - Recharts & Axios	- Gorgeous web HMI accessible via browser. - Real-time charts & interactive controls.
-------------------------------------	---	---	--

6. Sample Code Architectures

Python ML Inference & FastAPI Server (Snippet)

```

from fastapi import FastAPI, WebSocket
from fastapi.middleware.cors import CORSMiddleware
import joblib
import json

app = FastAPI()
# Enable CORS for React frontend
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_credentials=True, allow_methods=["*"], allow_headers=["*"])

# Load pre-trained machine learning model
ml_model = joblib.load("anomaly_detector.pkl")

# Active WebSocket connections
active_connections = []

@app.post("/api/telemetry")
async def receive_telemetry(data: dict):
    # Prepare data for ML model
    features = [[data["tank_level"], data["temperature"], data["gas_ppm"], data["current_draw"]]]

    # Run prediction
    prediction = int(ml_model.predict(features)[0]) # 0=Normal, 1=Anomaly, 2=Fault

    # Pack result
    payload = {
        "telemetry": data,
        "prediction": prediction,
        "status_label": ["Healthy", "Drift / Warning", "System Fault"][prediction]
    }

    # Broadcast to all active React web clients
    for connection in active_connections:
        await connection.send_text(json.dumps(payload))

    return {"status": "success", "prediction": prediction}

@app.websocket("/ws/live")
async def websocket_endpoint(websocket: WebSocket):

```

```
await websocket.accept()
active_connections.append(websocket)
try:
    while True:
        await websocket.receive_text() # Keep connection alive
except:
    active_connections.remove(websocket)
```